Vehicle Counting Using Classical Computer Vision

Overview

This project estimates the **total number of vehicles** appearing in a traffic video captured by a **static camera**, using **only classical computer vision techniques**. No deep learning models, object detectors, or learning-based approaches are used.

The solution is designed to be **robust**, **deterministic**, and **fully compliant** with the constraints of the Vehant Technologies Vehicle Count Hackathon. Special emphasis is placed on **engineering robustness**, **clarity of logic**, and **explainability of design choices**.

Problem Statement

Input

- A traffic video file (path provided to the solution)
- Video characteristics:
    - Static camera (no camera motion)
    - Vehicles moving predominantly away from the camera

Output

- A single integer representing the **total number of vehicles** visible in the video

No ground-truth annotation file is provided. Vehicle counts are determined directly by analyzing the video content.

Key Observations About the Data

The approach is built on the following observations about the provided traffic videos:

- The camera remains fixed throughout the video.
- Vehicles exhibit consistent motion in a dominant direction.
- The background remains largely unchanged over time.
- Vehicles can be distinguished as coherent moving regions.

These properties make the problem well-suited for motion-based classical computer vision methods rather than appearance-based object detection.

High-Level Methodology

The solution follows a structured classical vision pipeline:

1. Background subtraction to extract motion.
2. Morphological filtering to stabilize motion masks.
3. Connected components analysis to isolate vehicle-sized regions.
4. Motion direction estimation from early frames.
5. Dynamic placement of a counting line perpendicular to traffic flow.
6. Centroid-based tracking of moving regions.
7. Line-crossing logic to count vehicles exactly once.

Each stage is designed to be simple, interpretable, and robust.

Background Subtraction and Motion Extraction

A MOG2 background subtractor is used to separate moving objects from the static background. Since the camera does not move, background modeling remains stable over time.

To improve the quality of the foreground mask:

- Morphological opening removes small noise.
- Morphological closing fills gaps inside vehicle regions.

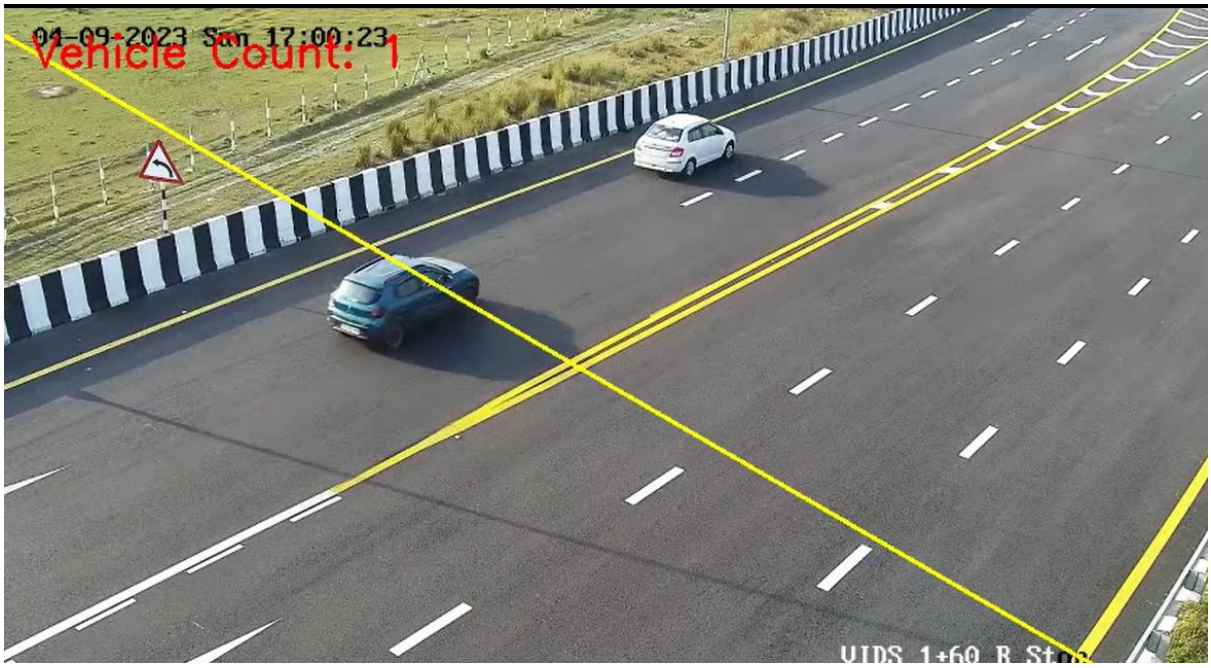
Connected components are then extracted from the binary mask. Components with an area below a frame-relative threshold are discarded to suppress noise and non-vehicle motion.

Motion Direction Estimation and Line Placement

Instead of assuming a fixed horizontal counting line, the algorithm estimates the **dominant direction of vehicle motion** directly from the video.

During an initial phase, the centroid of the largest moving region is tracked across several frames. The displacement of this centroid provides an estimate of the traffic flow direction.

The counting line is then placed **perpendicular** to this estimated direction. This makes the solution robust to camera tilt, road curvature, and perspective effects, and avoids hardcoding assumptions about video orientation.



Tracking and Counting Logic

Detected motion regions are tracked using centroid proximity across frames. Each track maintains a short memory of missed frames to handle temporary occlusions or segmentation noise.

A vehicle is counted **exactly once** when its centroid crosses the counting line. This is detected using a signed distance test, which identifies a change of side relative to the line.

This approach avoids:

- double counting,
- reliance on object re-identification,
- dependence on bounding box overlap heuristics.

Robustness Through Dual Independent Executions

Background subtraction and motion estimation can be sensitive to early-frame noise, sparse traffic, or initialization effects. To improve robustness, the complete pipeline is executed **twice**:

- First run: motion estimation using **100 frames**
- Second run: motion estimation using **500 frames**

Each run is executed in a **separate Python process**, ensuring:

- independent background models,
- no shared OpenCV state,
- no memory contamination between runs.

This design choice improves reliability while remaining fully deterministic and compliant with the hackathon rules.

Final Decision Strategy

Let:

- c100 be the count from the run with 100 motion-estimation frames
- c500 be the count from the run with 500 motion-estimation frames

The relative difference is computed as:

|c100 − c500| / max(c100, c500)

Decision rule:

- If the difference is **greater than 50%**, the **higher** count is selected (to avoid severe under-counting).
- Otherwise, the **lower** count is selected as a conservative estimate to avoid over-counting due to noise or blob fragmentation.

This rule is deterministic, video-agnostic, and based on observed behavior rather than hardcoded video-specific logic.

Submission File Structure

The submission folder contains the following files:

- **main.py**
  Entry point of the solution. Contains the required Solution class and the forward(video_path) method.
- **worker.py**
  Implements the full classical computer vision pipeline. Executes once per run and prints only the final vehicle count.
- **requirements.txt**
  Lists external Python dependencies.
- **README document**
  Explains the methodology, design choices, assumptions, and robustness strategy.

How the Evaluator Executes the Code

The evaluator will execute logic equivalent to:

- Import the Solution class from main.py
- Instantiate the class

- Call forward(video_path)
- Receive a single integer as output

No command-line arguments or manual steps are required during evaluation.

Dependencies

The solution depends only on the following external libraries:

- OpenCV (opencv-python)
- NumPy

All other imports are from the Python standard library.

Assumptions and Limitations

Assumptions

- The camera remains static.
- Vehicles move predominantly in a single direction.
- Vehicles are visually separable as moving regions.

Limitations

- Heavy occlusion may lead to under-counting.
- Very slow vehicles may partially blend into the background.
- Sudden lighting changes may temporarily affect foreground quality.

These limitations are inherent to classical background-subtraction-based approaches.

Compliance With Hackathon Rules

This solution fully complies with all stated rules:

- Uses only classical computer vision techniques
- Does not use deep learning or learning-based models
- Does not hardcode video-specific logic
- Produces deterministic and reproducible results
- Executes automatically without manual intervention
- Designed to be robust across different videos

Conclusion

This solution emphasizes **understanding the data**, **clear design choices**, and **robust engineering** rather than heuristic tuning or model complexity. The dual independent execution strategy improves reliability while remaining fully within the spirit and constraints of the challenge.
